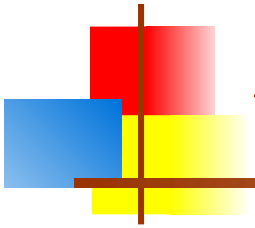


Git Essentials

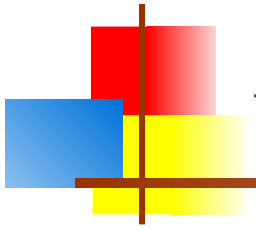
Basic use of **Git** for **cooperative work**

Adapted by Paulo Pedreiras from:
David Matuszek, CIT 590/591 Programming Languages & Techniques I, GIT slides



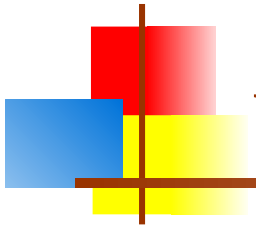
Version control systems

- Version control (or revision control, or source control) is all about managing multiple versions of documents, programs, web sites, etc.
  - Almost all “real” projects use some kind of version control
  - Essential for team projects, but also very useful for individual projects
- Some well-known version control systems are CVS, Subversion, Mercurial, and Git
  - CVS and Subversion use a “central” repository; users “check out” files, work on them, and “check them in”
  - Mercurial and Git treat all repositories as equal
- Distributed systems like Mercurial and Git are newer and are gradually replacing centralized systems like CVS and Subversion



Why version control?

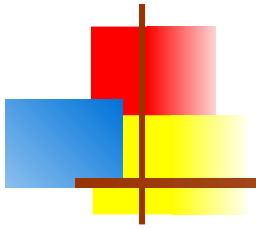
- For working by yourself:
  - Gives you a “time machine” for going back to earlier versions
  - Gives you great support for different versions (standalone, web app, etc.) of the same basic project
- **For working with others:**
  - **Greatly simplifies concurrent work, merging changes**
- For getting an internship or job:
  - Any company with a clue uses some kind of version control
  - Companies without a clue are bad places to work



Why Git?

- Git has many advantages over earlier systems such as CVS and Subversion
  - More efficient
  - Better workflow, etc.
    - See the literature for an extensive list of reasons
    - Of course, there are always those who disagree

- Best competitor: Mercurial
    - Same concepts, slightly simpler to use (for some)
    - Much less popular than Git

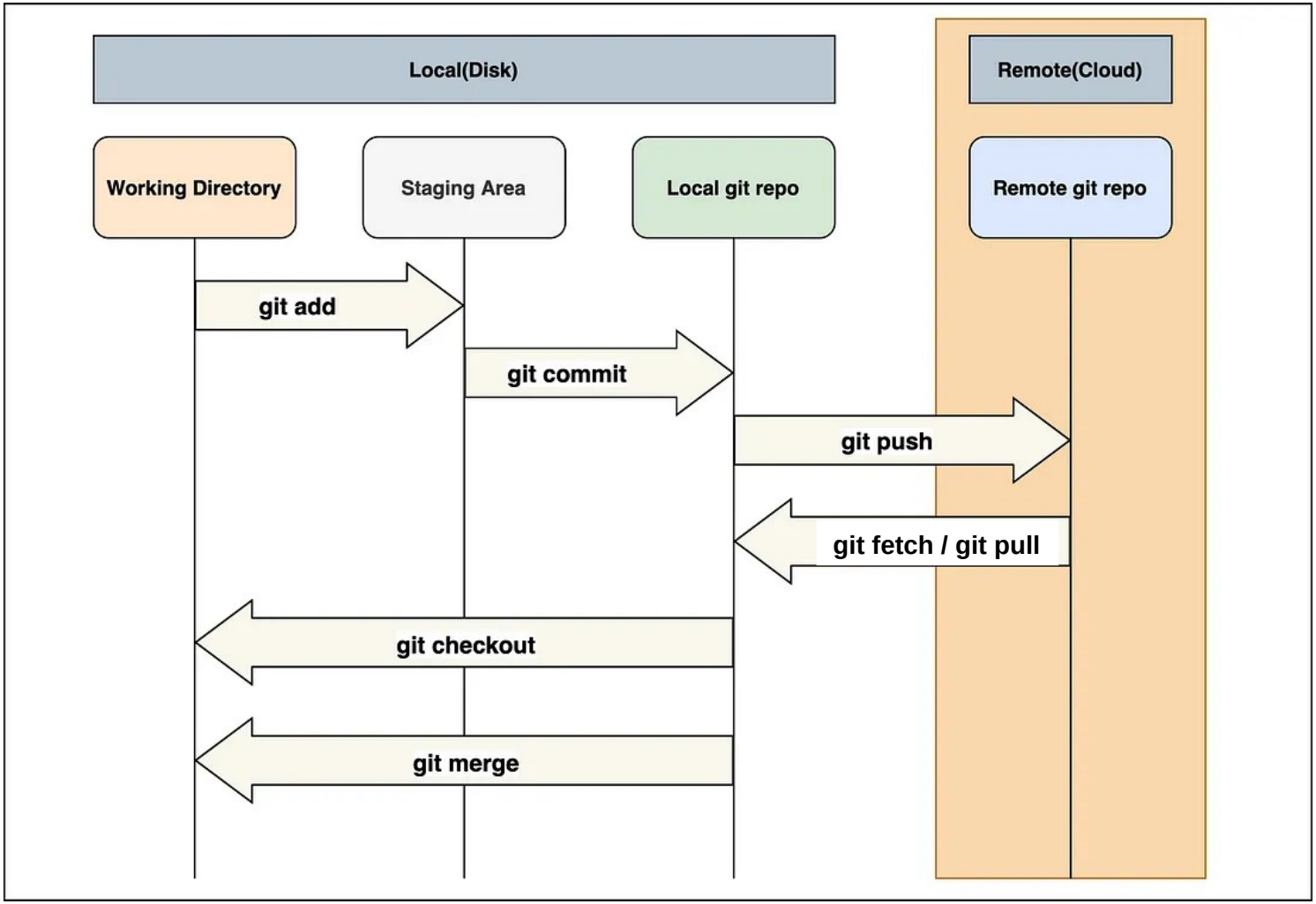


Documentation and integration

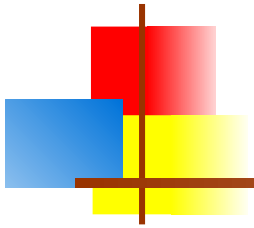
- There are many online materials that are helpful:
  - Official documentation: http://git-scm.com/doc
  - A very good tutorial: https://www.w3schools.com/git/default.asp?remote=github

- Note: **Git is primarily a command-line (CLI) tool**
  - Several IDEs already have an integrated git interface
  - E.g. VSCode, MPLabX, etc.
    - Easy and quick
    - Just hit a few buttons or use shortcuts
    - No need to type complex commands
  - But even in these cases it is important to master CLI git because sometimes the tools don’t do what you expect or planned

Overview



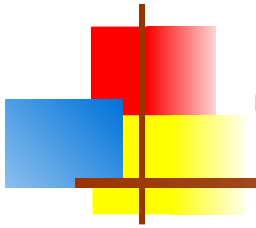
“Staging Area” is also know as “index”



General Git configuration

- Global configurations, required only once:
  - `git config --global user.name "John Doe"`
  - `git config --global user.email jd@xxx.yy`
  - `git config --global push.default simple`
    - `By default push only the current branch`
  - `git config --list`
    - `Check current configuration`

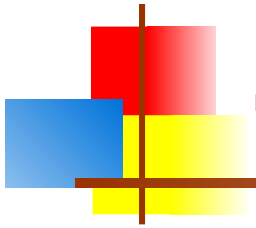
- Specific name and email address can be assigned to a particular project by going to the corresponding project directory and issuing the above commands without "`--global`"



The repository

- Your top-level **working directory** contains everything about your project
  - The working directory probably contains many subdirectories—source code, binaries, documentation, data files, etc.
  - One of these subdirectories, named .git, is your repository

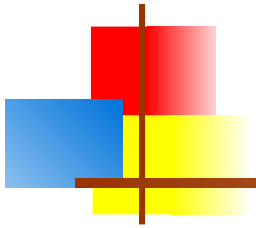
- At any time, you can take a "snapshot" of everything (or selected things) in your project directory, and put it in your repository
  - This "snapshot" is called a commit object
  - The commit object contains (1) a set of files, (2) references to the "parents" of the commit object, and (3) a unique "SHA1" name
  - Commit objects do not require huge amounts of memory: only differences are saved



The repository

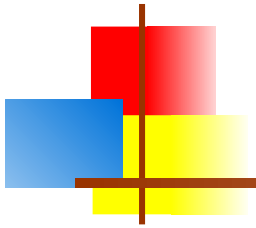
- You can work as much as you like in your working directory, but the repository isn't updated until you `commit` something

- When you typed `git init` in your project directory, or when you cloned an existing project, you have created a repository
  - The repository is a sub-folder named `.git` containing various files
  - The dot indicates a "hidden" directory
  - **You do *not* work directly with the contents of that directory; various git commands do that for you**



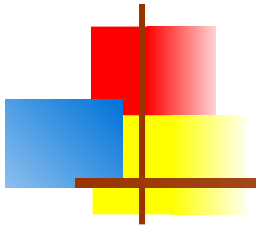
Making commits

- You do your work in your project folder, as usual
- If you create new files and/or folders, they are *not tracked* by Git unless you ask it to do so via the "`git add`" command. Examples:
  - *`git add newFile1 newFolder1 newFolder2 newFile2`*
  - *`git add .`* or *`git add *`*
    - Period signal means "this folder and subfolders", wildcards are allowed
    - There are other variants (git add -A, git add -i, …)
- Committing makes a "snapshot" of **everything being tracked** into your repository
  - A message telling what you have done is required
  - *`git commit –m "Message"`*
  - *`git commit`*
    - Opens an editor for you the enter the commit message
  - Commit messages should be concise but descriptive, as they are used to navigate the history of the work. E.g. "bugs fixed" is a terrible message, as usually many commits are related to bug fixes.



Making commits

- Staging and committing is so frequent that are some “typical commands”
  - *git add -A*
    - Add changes from all tracked and untracked files in the whole repository
  - *git commit -a –m "Commit message"*
    - “-a”: automatically stage files that have been modified and deleted
    - No need for “git add”
    - But note that new files (i.e. not previously staged) are not affected.
- Commits are “cheap” as only the differences are stored
  - However, “over-committing” is not a good practice as it makes navigation through the development history difficult
  - As everything in life ... use but do not abuse!

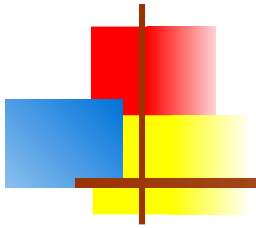


File states

A file in the working tree of a Git repository can have different states:

- untracked: the file is not tracked by the Git repository.
  - This means that the file never was staged nor committed.
- tracked: committed and not staged
- staged: staged to be included in the next commit
- dirty / modified: the file has changed but the change is not staged
- Use git status to check file state

Next slide: exercise!



Create and fill a repository

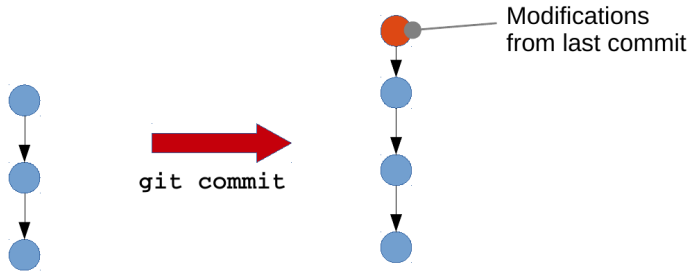
Practice:

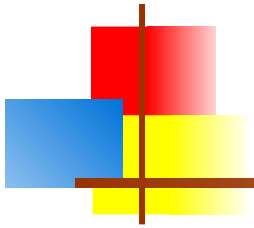
- Download the sample files from eLearning
- Init the git repo
- Configure git (if you haven't done it before)
- Check status
- Stage all files and make an initial commit
- Check status
- Edit file "module1.c" and modify the version to v2 (in the comments), adding also "v2" to printf()
- Check the status again
- Issue the following command
  - *git commit -m "Version 2"*
  - What was wrong? How to fix it?
    - Hit: check the status of the files

Commits and graphs

- When you commit your changes to git, it creates a commit object
  - A commit object represents the complete state of the project, including all the files in the project
  - The *very first* commit object has no "parents"
  - Usually, you take some commit object, make some changes, and create a new commit object; the original commit object is the parent of the new commit object
  - You can also merge two commit objects to form a new one
    - The new commit object has two parents

- Hence, commit objects form a **directed graph**
  - Git is all about using and manipulating this grap



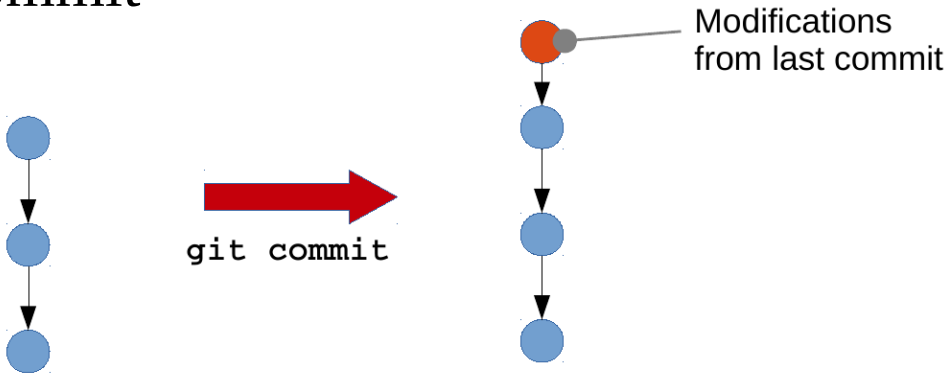


Commits and graphs

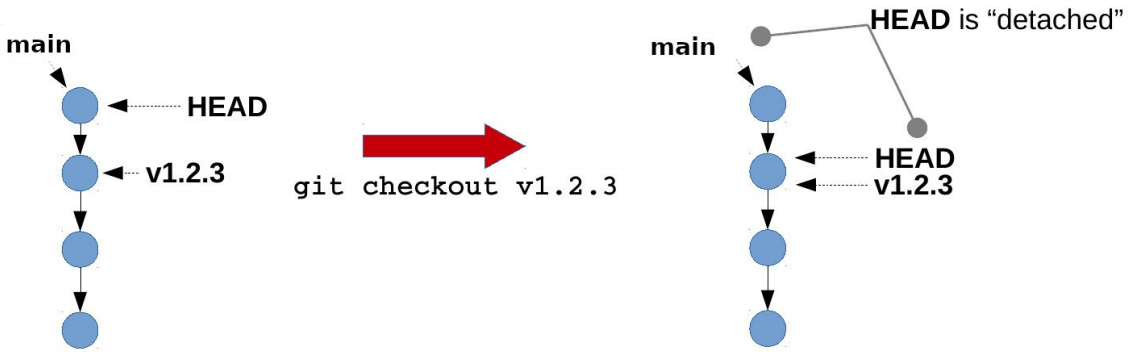
- A head is a reference to a commit object
- The current commit object is called HEAD (all caps)
- Usually, you will take HEAD (the current commit object), make some changes to it, and commit the changes, creating a new current commit object/HEAD
  - This results in a linear graph: A -> B -> C -> ...-> HEAD
- You can also take any previous commit object, make changes to it, and commit those changes
  - This creates a branch in the graph of commit objects
- You can merge any previous commit objects
  - This joins branches in the commit graph

Commits and graphs

Making a commit

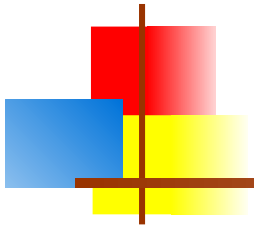


HEAD – the current commit object



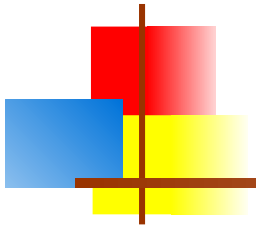
Working with commit objects

- git log : view the history of commits for a repository
  - git log --oneline --all --graph : all commits, summary only, with tree view
- git checkout 1234abcd : “1234abcd” is the abbreviated commit hash (6/7 bytes are enough)
  - Make sure current version is committed, otherwise changes will be lost
- git checkout master (or main): returns to the master tree
- Exercise
  - Recover the initial version (V1)
  - Confirm the version in the source code
  - Return to the last version



Checking older versions and using branches

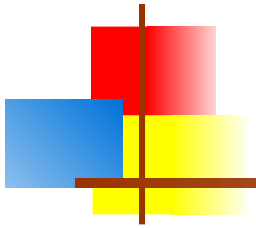
- git branch branch_name : creates a new branch named “branch_name”
- git branch [-a,-r]: lists branches (-r: remote, -a: local and remote)
- git checkout branch_name : switches to branch “branch_name”
- git merge branch_name : merges branch “branch_name” with current branch
- git -d/-D branch_name : deletes branch “branch_name”
- Exercise – create a branch of V1 named V1.1
  - Checkout version 1
  - Create a branch named bV1.1
  - Checkout branch bV1.1
  - Modify file “module1.c” (comments and printf) to generate V1.1
  - Commit bV1.1
  - Observe the commit tree with *git log --graph --oneline --all*
  - Switch back and forth between both branches
  - Merge bV1.1 with master!
    - As you can see, magic is limited …
    - Must edit the file and manually solve the conflicts (remove all conflict markers added by git, then commit)



git switch

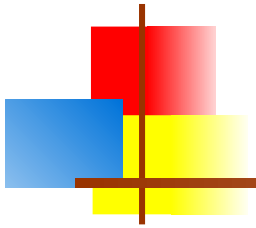
- In git v2.23 two commands were introduce: *git switch*, and *git restore*.
  - *git switch*: allow to create and navigate branches
  - *git restore*: allows to unstage file

- git switch is more convenient and easier to use than git branch and git checkout (which has too many options)
  - *git switch -c BRANCH_NAME*: creates a new branch and relocates the head to it
  - *git switch BRANCH_NAME*: relocates the head to BRANCH_NAME
  - *git switch -*: switch back to the previously checked out branch



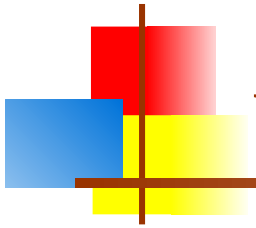
git switch

- To create a branch on a given commit object:
  - *git checkout COMMIT_OBJECT*
  - *git switch -c BRANCH_NAME*



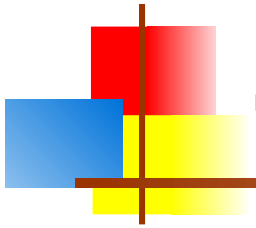
Clone a repository from elsewhere

- `git clone` *`URL`*
- `git clone` *`URL mypath`*
  - These make an exact copy of the repository at the given URL
- `git clone http://github.com/`*`user/project`*
  - Github is the most popular (free) public repository
  - Other popular public repositories are e.g. Gitlab and Bitbucket
- `git clone http://code.ua.pt/git/PRJ_REPO_ID`
  - For code.ua.pt projects
- All repositories are equal
  - But you can treat some particular repository (such as one on Github) as the "master"/"central" directory
- Typically, each team member works in his/her own repository, and "merges" with other repositories as appropriate



Working with others

- All repositories are equal, but it is convenient to have one central repository in the cloud
- Here's what you normally do:
  - Download the current HEAD from the central repository
  - Make your changes
  - Commit your changes to your local repository
  - Check to make sure someone else on your team hasn't updated the central repository since you got it
  - Upload your changes to the central repository
- If the central repository *has* changed since you got it:
  - It is *your* responsibility to **merge the two versions**
    - This is a strong incentive to commit and upload often!
  - Git can often do this for you, if there aren't incompatible changes



Typical workflow

- `git pull`
  - Get changes from a remote repository and merge them into your own repository
- `git status`
  - See what Git thinks is going on
  - Use this frequently!
- Work on your files (remember to `add` any new ones)
- `git commit -a –m “What I did”`
- `git push`

About rebasing

- **Rebasing** is the process of moving or combining a sequence of commits to a new base commit

  In some sense it is an alternative to git merge

- Can be used e.g. in the process of synchronizing a local repo with a remote one, as seen before
  (git pull --rebase)

- Can also be issued locally:

  Commit all changes to the branches of interest

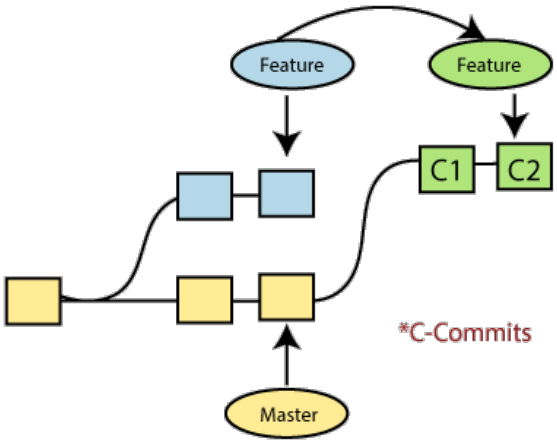
  Checkout the branch you want to rebase

  And then issue the command:

  - `git rebase <branch name>`

  Typical use case: You create a branch to make some experiments (test_branch) and now you want to integrate these changes on the main/master:

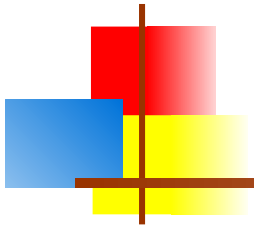
  - `git checkout test_branch`
  - `git rebase master`



https://www.javatpoint.com/git-rebase

Quick reference (1/2)

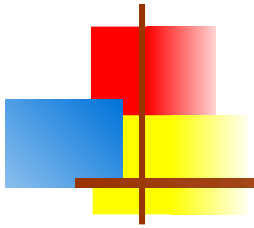
- git init : creates .git repository
- git add . / file_name(s) : tracks all/specific file(s)s in source directory
- git commit -a -m “Commit info” : commits all modified files to repository (including deleted files)
- git status : see what files have been modified, etc.
- git diff : see changes made to files
- git log : see list of commits
- git branch branch_name : creates a new branch named “branch_name”
- git checkout branch_name : selects branch “branch_name”
- git branch : lists branches
- git merge branch_name : merges branch “branch_name” with current branch



Quick reference (2/2)

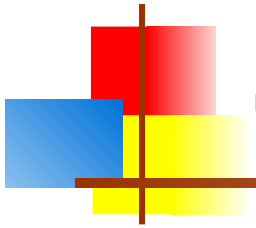
- git clone URL: Clones a repository into a newly created directory and creates remote-tracking branches for each branch in the cloned repository
- git push : update project on remote server
- git pull remote_repository : incorporates changes from a remote repository into the current branch

- Naming commits
  - Commits can be named to human-friendly names to facilitate the access
    - *git tag v2.5 1b2e1d63ff*
    - From now on you can use *git checkout v2.5*



Keeping it simple

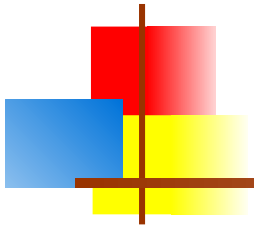
- If you:
  - Make sure you are current with the central repository
  - Make some improvements to your code
  - Update the central repository before anyone else does
- Then you don’t have to worry about resolving conflicts or working with multiple branches
  - All the complexity in git comes from dealing with these
- Therefore:
  - Make sure you are up-to-date before starting to work
  - Commit and update the central repository frequently
- If you need help: https://help.github.com/



The End

When I say I hate CVS with a passion, I have to also say that if there are any SVN [Subversion] users in the audience, you might want to leave. Because my hatred of CVS has meant that I see Subversion as being the most pointless project ever started. The slogan of Subversion for a while was "CVS done right", or something like that, and if you start with that kind of slogan, there's nowhere you can go. There is no way to do CVS right.

--Linus Torvalds, as quoted in Wikipedia



Bibliography

A few good tutorials on git:

- gittutorial - A tutorial introduction to Git, Git SCM homepage
  - https://git-scm.com/docs/gittutorial
- Git Tutorial, W3Schools
  - https://www.w3schools.com/git/
- Git Tutorial, Tutorialspoint
  - https://www.tutorialspoint.com/git/index.htm